

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 628 — Computational Biology

The data is noisy, the truth is often unavailable, and the algorithm has to say so

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 12 (with CSCE 640 and CSCE 717)

Prerequisites: CSCE 629 Analysis of Algorithms, especially dynamic programming; CSCE 633 Machine Learning and CSCE 676 Data Mining for Modules 10 and 11; no biology is assumed and Module 01 supplies what is needed

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A complete analysis of a public dataset you did not generate — with the pipeline versioned and reproducible, the multiple testing handled explicitly, at least one negative control, and a stated account of what the result would look like if the finding were an artefact

Course description

This is an algorithms course whose inputs come from an experiment somebody else ran. Module 01 establishes the biology you need and the position the course takes — which is that the hard part is rarely the algorithm and is almost always the gap between what was measured and what is being claimed.

The first half is the classical algorithmic core, and it is genuinely beautiful. Alignment is dynamic programming at its clearest (Modules 02 and 03); indexing and read mapping are where the data volume forces real engineering (Modules 04 and 05); and hidden Markov models and RNA folding are dynamic programming on richer structures (Modules 07 and 08).

The second theme is that every one of these produces a score, and a score is not a finding. Module 03 is about what an alignment score means — which requires a null model — and that question recurs in every later module, because the whole subject runs on statistics applied to measurements with structure the null model does not have.

The third is the one that distinguishes this from a pure algorithms course. Modules 10 and 11 are about multiple testing, batch effects, and confounding — where the computation is easy and the inference is where everything goes wrong — and Module 12 is about pipelines, because an unreproducible result is not a result.

And the closing position is about claims. Module 13 asks what a computational biology result establishes — because 'we identified a gene associated with the phenotype' is a statement about a statistical model applied to a particular cohort, and the distance between that and the biological claim is

the thing this course is trying to make visible.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 State the biological objects and what is actually measured.
- 2 Derive and implement sequence alignment algorithms.
- 3 Explain alignment scores, null models, and significance.
- 4 Explain indexing and heuristic database search.
- 5 Explain read mapping and assembly, and where they fail.
- 6 Build multiple alignments and phylogenies, with their error.
- 7 Apply hidden Markov models to biological sequence.
- 8 Predict RNA structure by dynamic programming.
- 9 Explain protein structure prediction and what changed.
- 10 Analyse expression data with multiple testing handled.
- 11 Explain association studies and their confounders.
- 12 Build a reproducible pipeline, and say why it matters.
- 13 Claim a biological result honestly.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	What Is Actually Measured	What did the machine actually produce?
2	Sequence Alignment	How similar are two strings, and where?
3	Scores and Significance	The score is 412. Is that good?
4	Search and Indexing	How do you align against forty billion bases?
5	Mapping and Assembly	Why can't you just put the pieces back together?
6	Multiple Alignment and Phylogeny	How confident is that branch?
7	Hidden Markov Models	Which parts of this sequence are which?
8	RNA Structure	What shape does this sequence fold into?

Wk	Module	Core question it answers
9	Protein Structure	Does a predicted structure tell you what the protein does?
10	Expression and Multiple Testing	You tested twenty thousand genes. How many are significant by chance?
11	Association and Confounding	The variant is associated. Does it do anything?
12	Pipelines and Reproducibility	Can you rerun your own analysis from last year?
13	Claiming a Biological Result	You found a gene. What do you actually know?

Projects

Project 1 — Write the alignment engine (after Module 6)

Implement the core algorithms, and find out what the scores actually mean.

Requirements

- **Global and local alignment implemented from the recurrence** — Needleman-Wunsch and Smith-Waterman — **with affine gap penalties**, and with traceback.
- **Verified against a library implementation** on the same inputs, with any disagreement explained rather than patched.
- **A substitution matrix used and its effect measured** (Module 03) — the same pair aligned under two matrices, and the difference reported.
- **An empirical null distribution: align shuffled sequences several thousand times and plot the score distribution.**
- **Then report where your real alignment's score falls in it**, as a measured rather than an assumed significance.
- **And the runtime measured against sequence length**, with the quadratic confirmed.

Done means

- **The empirical null built and used** — **which is the point of the project**, because **a score without a null is not interpretable** (Module 03 §2).
- **The affine gap recurrence correct**, which is where implementations usually break — and the three-matrix formulation is the one to get right.
- **The two substitution matrices giving different alignments**, reported as a finding about the method rather than as a nuisance.
- **And the disagreement with the library explained** — **usually it is a tie-breaking or a gap-at-the-end convention**, and tracking it down teaches more than agreement would.

Project 2 — A real dataset, honestly analysed (after Module 12)

Take public data you did not generate, and find out what it will and will not support.

Requirements

- **A public dataset you did not generate** — expression, variants, or sequence — **with its original paper read**, including the methods section.
- **A reproducible pipeline** (Module 12): **pinned versions, a named reference build, and a single command that reruns everything.**
- **Multiple testing handled explicitly** (Module 10 §3) — **with the number of tests stated and the correction named.**

- **At least one negative control: permuted labels, or a comparison that should produce nothing** — and its result reported whatever it is.
- **The batch and confounding structure examined** (Modules 10 §4, 11 §2) — or stated as unexaminable from the available metadata, which is a common and honest finding.
- **And the honest claim** in Module 13 §3's form, including what the result would look like if it were an artefact.

Done means

- **The negative control run and reported** — **a project with no negative control fails**, whatever it found, for Module 13 §2's reasons.
- **The pipeline rerunnable by somebody else**, from the repository, which is tested rather than asserted.
- **The multiple testing count honest** — **including the tests you ran and discarded**, which is the part everybody omits (Module 10 §3).
- **And the artefact account specific: 'if this were batch structure it would look like X, and here is whether it does'** — which is the analytical skill the whole course is aimed at.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Durbin, Eddy, Krogh & Mitchison — Biological Sequence Analysis	The primary text for Modules 02 through 08. Probabilistic models of sequence, done properly, and the HMM chapters are the best treatment anywhere. Library copy.

Resource	Role
Jones & Pevzner — An Introduction to Bioinformatics Algorithms	Modules 02, 04, 05, and 08. Algorithm-first, with the biology supplied as needed — which suits this course's audience exactly. Library copy.
Compeau & Pevzner — Bioinformatics Algorithms, with the free Rosalind problems	The exercises for Modules 02 through 08, free. Implementing these is the fastest way to find out whether you understood the recurrence.
Lander & Weinberg and the open genomics literature (free)	Module 01's biology and Modules 10 to 13's practice. PubMed Central is open access and is where the methods sections actually are.
Leek & Storey, and the batch effects literature (free)	Module 10. Why a large fraction of published expression findings were batch structure, and what to do about it.
The Bioconductor and Biopython documentation (free)	Modules 10 and 12. The standard tooling, and Bioconductor's statistical packages encode a great deal of hard-won practice.

Tooling

- **Python with biopython, or R with Bioconductor — and R for anything statistical**, because **the multiple-testing and differential-expression methods of Module 10 are implemented correctly there** and are easy to reimplement incorrectly.
- **But implement the core algorithms yourself first — Needleman-Wunsch, Viterbi, and Nussinov are each under fifty lines, and calling a library before writing one leaves the recurrence unlearned** (Project 1).
- **Public data, and plenty of it — NCBI, Ensembl, UniProt, GEO, and the 1000 Genomes data are all free — and Project 2 requires a dataset you did not generate**, which is the realistic case.
- **A workflow manager — Snakemake or Nextflow — and Module 12 §2 explains why a shell script is not sufficient** once a reference genome version is involved.
- **Containers, and a pinned environment**, because **'it worked last year'** is the characteristic failure here and the tools move faster than the papers.
- **And a negative control, always — permuted labels, a scrambled sequence, a known-null comparison — which is Module 13 §2's requirement and the cheapest insurance in the course.**

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.